在语言级虚拟机中，最核心的设计之一是：**指令如何操作数据**。

根据数据的组织与访问方式不同，语言虚拟机通常可以分为**栈虚拟机**和**寄存器虚拟机**两类。

栈虚拟机（Stack-based VM）

栈虚拟机以**操作数栈**作为主要的数据通道，指令通常**不显式指定操作数位置**，而是隐式地从栈顶取值并将结果压回栈中。

例如，计算表达式 `a + b` 时，其执行过程可能为：

1. 将 `a` 压栈
2. 将 `b` 压栈
3. 执行 `ADD` 指令，从栈顶弹出两个值，相加后再压栈

在这种模型中：

- 指令格式简单
- 不需要在指令中编码寄存器或地址
- 执行状态高度依赖栈内容

典型代表：JVM、早期 Python VM。

寄存器虚拟机（Register-based VM）

寄存器虚拟机在虚拟层面引入了一组**虚拟寄存器**，指令显式指定操作数与结果寄存器。

同样是计算 `a + b`，其形式可能为：

`ADD R1, R2, R3 ; R1 = R2 + R3`

在这种模型中：

- 指令显式描述数据流

- 执行状态分布在寄存器集合中
- 更接近真实 CPU 的编程模型

典型代表：Lua VM、Dalvik VM。

与真实计算机的区别

尽管名称相似，**语言虚拟机中的“栈”和“寄存器”并不等同于真实硬件**。

1. 虚拟寄存器不是 CPU 寄存器

- 虚拟寄存器通常只是内存中的一段数组或结构体
- 数量、宽度、语义完全由虚拟机设计者决定
- 不受硬件架构约束

例如，在 Lua VM 中，所谓“寄存器”本质上是**栈帧中的槽位（slot）**。

2. 虚拟机的栈不是硬件调用栈

- 语言虚拟机的栈用于保存**语言层面的中间值与状态**
- 与 CPU 的调用栈可以完全无关
- 一个虚拟机栈帧，往往映射到多个甚至零个真实栈帧

这也是为什么在 VM 程序中，**控制流与调用关系往往难以通过传统栈回溯理解**。

3. 指令语义是人为设计的

真实 CPU 的指令集受限于硬件实现，而虚拟机指令集是**人为定义的抽象语义**：

- 一条 VM 指令可以对应多条机器指令
- 也可以携带高层语义（如对象访问、类型检查）

- 甚至可以故意设计为反直觉（VMP 常见做法）